CCS335 – Cloud Computing Lab

Combined Lab Preparation Answer

Prepared by combining the original exam paper with the improved content supplied in the attached text.

EXPERIMENT 1: Install a C Compiler in a Virtual Machine and Execute a Fibonacci Series Program Using C

Aim

To install and configure the GCC (GNU Compiler Collection) in a Linux-based virtual machine and develop, compile and execute a C program to generate the Fibonacci series for a given number of terms.

Procedure

1. Start the host computer and open virtualization software such as Oracle VirtualBox or VMware Workstation.
2. Start the Linux virtual machine and wait until the operating system is completely loaded.
3. Log in using the username and password configured during Linux installation.
4. Open the Terminal application to execute Linux commands.
5. Update the package repository using sudo apt update to obtain the latest available package information.
6. Install the GCC compiler using the command sudo apt install gcc -y.
7. Verify the installation by executing gcc --version. The installed compiler version will be displayed.
8. Create a new C source file named fibonacci.c using a text editor such as Nano or Gedit.
9. Enter the C program to generate the Fibonacci series and save the source file.
10. Open the terminal in the directory containing the source file.
11. Compile the program using gcc fibonacci.c -o fibonacci. This generates an executable file named fibonacci.
12. Execute the program using ./fibonacci and enter the required number of terms.
13. Observe the generated Fibonacci sequence and verify that every term is the sum of the previous two terms.
14. Repeat the execution with a different number of terms to confirm that the program produces the expected result.

Program / Commands

Install and verify GCC: sudo apt update sudo apt install gcc -y gcc --version Program – fibonacci.c: #include int main() { int n, a = 0, b = 1, c, i; printf("Enter the number of terms: "); scanf("%d", &n;); printf("Fibonacci Series:\n"); for(i = 1; i <= n; i++) { printf("%d ", a); c = a + b; a = b; b = c; } return 0; } Compile and execute: gcc fibonacci.c -o fibonacci ./fibonacci

Output of Program

Sample Input: Enter the number of terms: 10 Expected Output: Fibonacci Series: 0 1 1 2 3 5 8 13 21 34 Explanation: The Fibonacci series starts with 0 and 1. Every subsequent number is calculated by adding the previous two numbers. The program successfully generates the requested number of terms.

Result

Thus, the GCC compiler was successfully installed and configured in the Linux virtual machine. The C program for generating the Fibonacci series was written, compiled and executed successfully, and the expected sequence was obtained.

EXPERIMENT 2: Write a Google App Engine Program to Generate Nth Odd Numbers and Deploy It to Google Cloud

Aim

To develop a Python-based web application using the Flask framework that generates the first N odd numbers and deploy the application on Google App Engine, a Platform as a Service (PaaS) offered by Google Cloud.

Procedure

1. Open a web browser and navigate to the Google Cloud Console.
2. Sign in using a valid Google account and create a new Google Cloud project.
3. Select the newly created project and enable the required Google Cloud services.
4. Enable billing if it is required for the selected deployment configuration.
5. Install Python and the Google Cloud CLI on the local computer.
6. Open the terminal or command prompt and execute gcloud init to initialize the Google Cloud CLI.
7. Authenticate using the Google account and select the appropriate Google Cloud project.
8. Create a new application directory named OddNumbers to store the project files.
9. Inside the directory, create a Python file named main.py.
10. Import the Flask module and create a Flask application instance.
11. Define a route that accepts the value N from the user's browser input.
12. Implement the logic to generate odd numbers using the mathematical expression 2 × i + 1.
13. Add input validation to ensure that N is a valid positive integer.
14. Create a file named requirements.txt and include Flask as the application dependency.
15. Create an app.yaml file and specify the supported Python runtime.
16. Open the terminal inside the application directory and execute gcloud app deploy.
17. Wait until the deployment process completes successfully.
18. Execute gcloud app browse to open the deployed application in the browser.
19. Enter a sample value such as 5 and verify that the first five odd numbers are displayed correctly.

Program / Commands

File – main.py: from flask import Flask, request app = Flask(__name__) @app.route("/") def odd_numbers(): n = request.args.get("n") if n is None: return ''' Generate Odd Numbers Enter N: ''' try: n = int(n) if n 1000: return "Enter a number between 1 and 1000" result = [2 * i + 1 for i in range(n)] return f" First {n} Odd Numbers {result}" except ValueError: return "Enter a valid integer" if __name__ == "__main__": app.run(host="0.0.0.0", port=8080) CCS335 – Cloud Computing Lab Page 5 File – requirements.txt: Flask File – app.yaml: runtime: python313 Deployment commands: gcloud init gcloud app deploy gcloud app browse

Output of Program

Input: N = 5 Expected Browser Output: First 5 Odd Numbers [1, 3, 5, 7, 9] Explanation: The application accepts the number of terms from the user and generates the first N odd numbers. The odd numbers are calculated using the formula: [Odd(i)=2i+1] The application is hosted on Google App Engine and can be accessed through its deployed URL.

Result

Thus, a Python web application was successfully developed using Flask to generate the first N odd numbers. The application was deployed on Google App Engine and tested through a web browser, producing the expected output.

EXPERIMENT 3: Create a Virtual Machine Using a Hypervisor and Install Linux

Aim

To create and configure a virtual machine using a hypervisor such as Oracle VirtualBox or VMware Workstation and install a Linux operating system in the virtual environment.

Procedure

1. Start the host computer and ensure that hardware virtualization is enabled in the BIOS/UEFI settings if required.
2. Download Oracle VirtualBox or VMware Workstation from its official website.
3. Install the virtualization software by following the installation wizard.
4. Download a Linux distribution ISO image, such as Ubuntu, from its official website.
5. Open the installed virtualization software and select the New option to create a virtual machine.
6. Enter a suitable name for the virtual machine, such as Ubuntu Linux.
7. Select Linux as the operating system type and choose the appropriate Ubuntu version.
8. Allocate sufficient RAM to the virtual machine based on the host computer's available memory.
9. Create a new virtual hard disk and allocate sufficient storage space, preferably around 25 GB or more.
10. Select the appropriate virtual hard disk format and choose dynamically allocated storage if required.
11. Open the virtual machine settings and attach the downloaded Linux ISO image to the virtual optical drive.
12. Start the virtual machine. The system will boot from the ISO image and display the Linux installation screen.
13. Select the language, keyboard layout and installation preferences.
14. Choose the installation option and configure the virtual disk according to the installation requirements.
15. Enter the required username, computer name and password.
16. Wait until the installation process is completed. The installer copies the required operating system files into the virtual hard disk.
17. Restart the virtual machine after installation and remove the installation ISO when prompted.
18. Log in using the previously created username and password.
19. Open the Linux desktop and verify that the operating system is functioning correctly.
20. Open the terminal and execute basic commands such as pwd, ls and uname -a to verify the installation.

Program / Commands

Not required. This experiment is performed using the specified software tools and commands.

Output of Program

Expected Output: - The VirtualBox or VMware window displays the newly created Linux virtual machine. - The Ubuntu login screen appears after installation. - After successful login, the Linux desktop environment is displayed. - Basic Linux commands execute successfully in the terminal. Sample Terminal Output: $ pwd /home/user $ uname -s Linux Explanation: The Linux operating system runs inside the virtual machine without replacing the host operating system. The virtual machine uses the hardware resources allocated by the hypervisor.

Result

Thus, a virtual machine was successfully created using a hypervisor, and the Linux operating system was installed and configured in the virtual environment. The installation was verified by accessing the Linux desktop and executing basic terminal commands.

EXPERIMENT 4: Set Up a One-Node Hadoop Cluster

Aim

To install and configure Apache Hadoop in pseudo-distributed mode on a Linux system and establish a single-node Hadoop cluster for processing distributed data using HDFS and YARN.

Procedure

1. Start the Linux virtual machine and log in using the configured user account.
2. Open the terminal and update the package repository using sudo apt update.
3. Install a compatible Java Development Kit because Hadoop requires Java to execute its services.
4. Verify the Java installation using java -version.
5. Install the OpenSSH server package using the appropriate package manager.
6. Generate an SSH key pair using ssh-keygen and configure passwordless SSH access to localhost.
7. Download a compatible Apache Hadoop binary distribution from the official Apache Hadoop website.
8. Extract the downloaded archive into the home directory and identify the Hadoop installation path.
9. Configure the JAVA_HOME and HADOOP_HOME environment variables in the shell configuration file.
10. Add Hadoop's bin and sbin directories to the system PATH.
11. Reload the shell configuration using source ~/.bashrc and verify the Hadoop installation using hadoop version.
12. Open the Hadoop configuration directory and edit core-site.xml to specify the default HDFS filesystem URI.
13. Edit hdfs-site.xml and configure the replication factor as 1 because only one DataNode is being used.
14. Configure mapred-site.xml to use YARN as the execution framework.
15. Edit yarn-site.xml and enable the MapReduce shuffle service.
16. Save all configuration files and verify that their XML structure is correct.
17. Format the NameNode using hdfs namenode -format to initialize the HDFS metadata.
18. Start the HDFS services using start-dfs.sh.
19. Start the YARN services using start-yarn.sh.
20. Execute the jps command to verify that all required Hadoop Java processes are running.
21. Open the NameNode web interface using http://localhost:9870 to inspect the HDFS cluster.
22. Open the ResourceManager web interface using http://localhost:8088 to inspect the YARN resource management service.

Program / Commands

Install packages and configure SSH: sudo apt update sudo apt install openjdk-11-jdk openssh-server -y ssh-keygen -t rsa -P "" -f ~/.ssh/id_rsa cat ~/.ssh/id_rsa.pub >> ~/.ssh/authorized_keys Example environment variables (add to ~/.bashrc): export HADOOP_HOME=$HOME/hadoop-3.3.6 export JAVA_HOME=/usr/lib/jvm/java-11-openjdk-amd64 export PATH=$PATH:$HADOOP_HOME/bin:$HADOOP_HOME/sbin source ~/.bashrc core-site.xml: fs.defaultFS hdfs://localhost:9000 hdfs-site.xml: dfs.replication 1 CCS335 – Cloud Computing Lab Page 8 mapred-site.xml: mapreduce.framework.name yarn yarn-site.xml: yarn.nodemanager.aux-services mapreduce_shuffle Start and verify: hdfs namenode -format start-dfs.sh start-yarn.sh jps

Output of Program

Expected Output of jps: NameNode DataNode SecondaryNameNode ResourceManager NodeManager Jps Web Interfaces: - NameNode: http://localhost:9870 - ResourceManager: http://localhost:8088 Explanation: The NameNode manages the metadata of files stored in HDFS, while the DataNode stores the actual data blocks. YARN manages cluster resources and executes processing tasks. Since all services are running on the same machine, this configuration is known as a pseudo-distributed Hadoop cluster.

Result

Thus, Apache Hadoop was successfully installed and configured in pseudo-distributed mode. The NameNode, DataNode, SecondaryNameNode, ResourceManager and NodeManager services were started and verified, establishing a functional single-node Hadoop cluster.

EXPERIMENT 5: Simulate a Cloud Scenario Using CloudSim and Run a Custom Scheduling Algorithm

Aim

To simulate a cloud computing environment using the CloudSim toolkit and implement a custom Shortest Cloudlet First (SCF) scheduling algorithm to arrange cloudlets according to their execution lengths.

Procedure

1. Install a compatible Java Development Kit on the computer and verify the installation using java -version.
2. Download the CloudSim toolkit from its official source and extract the package.
3. Open Eclipse or another Java-supported integrated development environment.
4. Create a new Java project named CloudSimSimulation.
5. Add the CloudSim libraries and required dependencies to the project's build path.
6. Create a Java class to initialize the CloudSim simulation environment.
7. Configure the simulation by specifying the number of users, calendar and trace settings.
8. Create a Datacenter to represent the cloud infrastructure.
9. Define the required hosts and processing elements with suitable CPU, RAM, storage and bandwidth values.
10. Create a DatacenterBroker to manage communication between the user and the cloud datacenter.
11. Create virtual machines with appropriate processing capacity, memory, bandwidth and storage configurations.
12. Create multiple cloudlets representing user-submitted computing tasks.
13. Assign different execution lengths to the cloudlets so that the scheduling algorithm can arrange them.
14. Implement the Shortest Cloudlet First scheduling logic by sorting the cloudlets in ascending order of their execution lengths.
15. Submit the virtual machines and sorted cloudlet list to the broker.
16. Start the CloudSim simulation and allow the configured cloudlets to execute.
17. Retrieve the cloudlet execution details, including their IDs, status and execution information.
18. Display the scheduling order and simulation results in the console.
19. Stop the simulation and verify that the cloudlets have been processed according to the configured scheduling logic.

Program / Commands

Core SCF scheduling demonstration – SCFScheduler.java: import java.util.Arrays; public class SCFScheduler { public static void main(String[] args) { int[] cloudlets = {4000, 1000, 3000, 2000}; System.out.println("Before Scheduling:"); for (int c : cloudlets) { System.out.println(c); } // Sort cloudlets in ascending order of length Arrays.sort(cloudlets); System.out.println("\nAfter SCF Scheduling:"); for (int c : cloudlets) { System.out.println("Cloudlet Length: " + c); } } } Compile and execute: javac SCFScheduler.java java SCFScheduler

Output of Program

Input Cloudlet Lengths: 4000 1000 3000 2000 Output Before Scheduling: 4000 1000 3000 2000 Output After Shortest Cloudlet First Scheduling: Cloudlet Length: 1000 Cloudlet Length: 2000 Cloudlet Length: 3000 Cloudlet Length: 4000 Explanation: The SCF scheduling algorithm arranges the cloudlets in ascending order of execution length. The cloudlet requiring the least execution time is placed first, followed by the remaining cloudlets. This demonstrates the basic scheduling order. A complete CloudSim experiment should connect the sorted cloudlets to the broker and execute them within the configured datacenter simulation.

Result

Thus, a cloud computing scenario was demonstrated using the CloudSim toolkit. The Shortest Cloudlet First scheduling algorithm was implemented to arrange cloudlets based on their execution lengths, and the expected scheduling order was obtained.

EXPERIMENT 6: Install Windows in VirtualBox and Find the Largest Element in an Array Using C

Aim

To create a Windows virtual machine using Oracle VirtualBox, install the Windows operating system and execute a C program to identify the largest element in a given array.

Procedure

1. Download and install Oracle VirtualBox on the host computer.
2. Download a compatible Windows ISO image from the official Microsoft website.
3. Open VirtualBox and click the New option to create a virtual machine.
4. Enter the virtual machine name and select Microsoft Windows as the operating system type.
5. Allocate sufficient RAM and processor resources to the virtual machine.
6. Create a virtual hard disk with adequate storage space for the Windows installation.
7. Attach the downloaded Windows ISO image to the virtual optical drive.
8. Start the virtual machine and wait for the Windows installation screen to appear.
9. Select the language, time format and keyboard layout.
10. Follow the Windows installation wizard and complete the operating system setup.
11. Create a user account and configure the initial Windows settings.
12. Restart the virtual machine and verify that Windows starts successfully.
13. Install a C compiler such as MinGW-w64 or an IDE that includes a GCC compiler.
14. Open the compiler terminal or command prompt and create a file named largest.c.
15. Enter the C program to read the array size and its elements from the user.
16. Initialize the maximum variable with the first array element.
17. Compare each remaining array element with the current maximum and update it whenever a larger value is found.
18. Compile the program using GCC and execute the generated executable file.
19. Enter the array size and sample values when prompted.
20. Observe the output and verify that the program displays the largest element correctly.

Program / Commands

Program – largest.c: #include int main() { int a[100], n, i, max; printf("Enter number of elements: "); scanf("%d", &n;); if (n 100) { printf("Invalid array size"); return 1; } printf("Enter array elements:\n"); for (i = 0; i max) { max = a[i]; } } printf("Largest element = %d\n", max); return 0; } Compile and execute in the compiler environment: gcc largest.c -o largest.exe CCS335 – Cloud Computing Lab Page 12 largest.exe

Output of Program

Sample Input: Enter number of elements: 5 Enter array elements: 10 25 45 12 30 Expected Output: Largest element = 45 Explanation: The program reads all elements of the array and compares them one by one. The maximum variable stores the largest value encountered during the comparison process. After all elements have been checked, the largest value is displayed.

Result

Thus, Windows was successfully installed in Oracle VirtualBox, and a C compiler was configured in the virtual environment. The C program to find the largest element in an array was compiled and executed successfully, producing the correct result.

EXPERIMENT 7: Install Google App Engine and Create a Python Temperature Converter Web Application

Aim

To develop a Python-based temperature converter web application using the Flask framework and deploy it on Google App Engine to convert temperature values from Celsius to Fahrenheit.

Procedure

1. Open a web browser and navigate to the Google Cloud Console.
2. Sign in using a valid Google account and create a new Google Cloud project.
3. Select the created project and enable the required Google Cloud services.
4. Enable billing if required by the selected App Engine configuration.
5. Install Python on the local computer and verify its installation using python --version.
6. Install the Google Cloud CLI and initialize it using the gcloud init command.
7. Authenticate using the Google account and select the appropriate cloud project.
8. Create a project directory named TemperatureConverter.
9. Inside the directory, create a Python source file named main.py.
10. Import the Flask module and create a Flask application instance.
11. Define a web route that displays an HTML form for accepting the Celsius temperature from the user.
12. Retrieve the entered temperature using the request object provided by Flask.
13. Convert the entered value into a floating-point number to support decimal temperature values.
14. Apply the Celsius-to-Fahrenheit conversion formula:

[

F=\frac{9}{5}C+32

]

1. Display the converted Fahrenheit value on the web page.
2. Add input validation to handle invalid or non-numeric values.
3. Create a requirements.txt file and specify Flask as the required dependency.
4. Create an app.yaml file and specify the supported Python runtime.
5. Open the terminal inside the application directory and execute gcloud app deploy.
6. Wait until the deployment process completes successfully.
7. Execute gcloud app browse to open the deployed web application.
8. Enter a Celsius value such as 25 and verify the corresponding Fahrenheit value.

Program / Commands

File – main.py: from flask import Flask, request app = Flask(__name__) @app.route("/") def home(): c = request.args.get("c") if c is None: return '" Temperature Converter Enter Celsius: '" try: celsius = float(c) fahrenheit = (celsius * 9 / 5) + 32 return (f" Temperature Conversion " f"Celsius: {celsius} " f"Fahrenheit: {fahrenheit:.2f}") except ValueError: return "Enter a valid temperature" if __name__ == "__main__": app.run(host="0.0.0.0", port=8080) File – requirements.txt: CCS335 – Cloud Computing Lab Page 14 Flask File – app.yaml: runtime: python313 Deployment commands: gcloud init gcloud app deploy gcloud app browse

Output of Program

Input: Celsius = 25 Expected Browser Output: Temperature Conversion Celsius: 25.0 Fahrenheit: 77.00 Explanation: The application receives a Celsius value from the user and applies the conversion formula to calculate its equivalent Fahrenheit temperature. For example: [F=\frac{9}{5}\times25+32=77^\circ F] The converted temperature is displayed on the deployed web application.

Result

Thus, a temperature converter web application was successfully developed using Python and Flask. The application was deployed on Google App Engine and tested through a browser, successfully converting Celsius values into Fahrenheit.

EXPERIMENT 8: Create a Virtual Machine and Transfer Files Between Host and Guest Using Drag and Drop

Aim

To create a virtual machine using Oracle VirtualBox and configure the host-guest integration features to transfer files between the host and guest operating systems using the drag-and-drop facility.

Procedure

1. Download Oracle VirtualBox from its official website and install it on the host computer.
2. Download the ISO image of the required guest operating system.
3. Open VirtualBox and select the New option to create a virtual machine.
4. Enter the virtual machine name and select the appropriate operating system type and version.
5. Allocate sufficient RAM, processor resources and virtual hard disk space.
6. Attach the guest operating system ISO image and start the virtual machine.
7. Complete the operating system installation and restart the virtual machine.
8. Log in to the guest operating system and verify that it is working correctly.
9. From the VirtualBox menu, select Devices → Insert Guest Additions CD Image.
10. Open the mounted Guest Additions image inside the guest operating system.
11. Execute the Guest Additions installer and follow the installation instructions.
12. Restart the guest operating system after completing the installation.
13. Shut down the virtual machine and open its settings from the VirtualBox Manager.
14. Navigate to General → Advanced and locate the Shared Clipboard and Drag and Drop options.
15. Set the Shared Clipboard option to Bidirectional if clipboard sharing is also required.
16. Set the Drag and Drop option to Bidirectional to enable file transfer in both directions.
17. Save the settings and restart the virtual machine.
18. Open the file manager inside the guest operating system.
19. Select a sample file from the host computer and drag it into the guest operating system.
20. Verify that the transferred file appears in the destination folder.
21. Repeat the process by dragging a file from the guest operating system to the host computer.
22. Open the transferred files and verify that their contents are intact.

Program / Commands

Not required. This experiment is performed using the specified software tools and commands.

Output of Program

Expected Output: - A file selected from the host operating system appears inside the guest operating system. - A file selected from the guest operating system can be transferred to the host operating system. - The transferred files are accessible from their respective destination folders. Verification: Host System: sample.txt Guest System: sample.txt File transfer completed successfully. Explanation: VirtualBox Guest Additions provides additional integration features between the host and guest systems. The bidirectional drag-and-drop setting allows files to be transferred between the two environments without using external storage devices.

Result

Thus, a virtual machine was successfully created using Oracle VirtualBox. Guest Additions was installed, and the bidirectional drag-and-drop feature was enabled to transfer files between the host and guest operating systems.

EXPERIMENT 9: Install a Hadoop Single-Node Cluster on Windows and Execute Word Count

Aim

To install and configure Apache Hadoop in a single-node environment on Microsoft Windows and execute the Word Count MapReduce application to count the frequency of words in an input text file.

Procedure

1. Start the Windows operating system and open a web browser.
2. Download and install a compatible Java Development Kit.
3. Configure the JAVA_HOME environment variable to point to the JDK installation directory.
4. Verify Java installation by executing java -version in the command prompt.
5. Download the Apache Hadoop binary distribution and extract it into a suitable directory, such as C:\hadoop.
6. Configure the HADOOP_HOME environment variable to point to the Hadoop installation directory.
7. Add the Hadoop bin directory to the system PATH.
8. Open the Hadoop configuration directory and edit the required configuration files.
9. Configure core-site.xml to specify the default filesystem URI.
10. Configure hdfs-site.xml to set the replication factor to 1 for the single-node cluster.
11. Configure mapred-site.xml to use YARN as the MapReduce execution framework.
12. Configure yarn-site.xml to enable the MapReduce shuffle service.
13. Configure the Windows-compatible Hadoop native binaries, including a version-compatible winutils.exe where required.
14. Create the required NameNode and DataNode storage directories.
15. Open the Hadoop command prompt and format the NameNode to initialize the HDFS filesystem.
16. Start the HDFS and YARN services using the Windows scripts provided by the Hadoop distribution.
17. Verify the running services and ensure that the cluster is ready to accept jobs.
18. Create a sample text file named input.txt containing a few lines of text.
19. Create an HDFS directory named /input using the hdfs dfs -mkdir command.
20. Upload the input file to HDFS using hdfs dfs -put input.txt /input.
21. Execute the Hadoop MapReduce Word Count example using the appropriate examples JAR file.
22. Specify /input as the input directory and /output as the output directory.
23. After successful execution, retrieve the result using hdfs dfs -cat /output/part-r-00000.
24. Observe the word-frequency output and verify that the number of occurrences is correct.

Program / Commands

Sample input file – input.txt: hello world hello hadoop hadoop is useful Hadoop Word Count command (adjust the JAR version/path as installed): hdfs dfs -mkdir -p /input hdfs dfs -put input.txt /input hadoop jar %HADOOP_HOME%\share\hadoop\mapreduce\hadoop-mapreduce-examples-3.3.6.jar wordcount /input /output hdfs dfs -cat /output/part-r-00000

Output of Program

Input File – input.txt: hello world hello hadoop hadoop is useful Expected Output: hadoop 2 hello 2 is 1 useful 1 world 1 Explanation: The Word Count application uses the MapReduce programming model. - Map Phase: Reads the input text and generates key-value pairs in which each word is associated with the value 1. - Reduce Phase: Groups identical words and adds their corresponding values to calculate the total frequency. For example, the word hello appears twice, so its final count is 2.

Result

Thus, Apache Hadoop was installed and configured in single-node mode on Windows. The input text file was uploaded to HDFS, and the Word Count MapReduce application was executed successfully to calculate the frequency of each word.

EXPERIMENT 10: Install Google App Engine SDK on Windows and Run a Hello World Application

Aim

To install the Google Cloud development tools on Microsoft Windows, create a simple Python-based Hello World web application using Flask and deploy the application using Google App Engine.

Procedure

1. Start the Windows operating system and open a web browser.
2. Navigate to the official Google Cloud website and locate the Google Cloud CLI download section.
3. Download the Google Cloud CLI installer suitable for Windows.
4. Execute the downloaded installer and follow the installation wizard.
5. Install Python if it is not already available on the system.
6. Open Command Prompt or Google Cloud SDK Shell.
7. Verify the installation by executing gcloud --version.
8. Initialize the Google Cloud CLI using the command gcloud init.
9. Sign in using a valid Google account and select the required Google Cloud project.
10. Enable billing if required by the selected App Engine configuration.
11. Create a new project directory named HelloWorld.
12. Inside the directory, create a Python file named main.py.
13. Import the Flask module and create a Flask application instance.
14. Define a route for the home page that returns the message Hello World!.
15. Create a file named requirements.txt and include Flask as the application dependency.
16. Create an app.yaml file and specify the supported Python runtime.
17. Save all the application files in the same project directory.
18. Open the command prompt inside the project directory.
19. Execute gcloud app deploy to upload and deploy the application to Google App Engine.
20. Wait for the deployment process to finish and check for successful deployment.
21. Execute gcloud app browse to open the deployed application in a web browser.
22. Verify that the browser displays the Hello World message.

Program / Commands

File – main.py: from flask import Flask app = Flask(__name__) @app.route("/") def hello(): return " Hello World! " if __name__ == "__main__": app.run(host="0.0.0.0", port=8080) File – requirements.txt: Flask File – app.yaml: runtime: python313 Deployment commands: gcloud init gcloud app deploy gcloud app browse

Output of Program

Expected Browser Output: Hello World! Expected Result: Application deployed successfully. Explanation: The Flask application handles requests made to the home page and returns a simple HTML response containing the Hello World message. Google App Engine hosts the application and makes it accessible through a web URL.

Result

Thus, the Google Cloud CLI was successfully installed and configured on Windows. A simple Hello World application was developed using Python and Flask, deployed on Google App Engine and accessed successfully through a web browser.

EXPERIMENT 11: Create and Execute Your First Container Using Docker

Aim

To install and configure Docker, download an Ubuntu image from Docker Hub, create a container using the downloaded image and execute commands inside the container.

Procedure

1. Start the operating system and open a web browser.
2. Visit the official Docker website and download Docker Desktop or Docker Engine according to the operating system.
3. Install Docker by following the installation instructions.
4. Start the Docker service or Docker Desktop and wait until the Docker engine becomes ready.
5. Open a terminal or command prompt to execute Docker commands.
6. Verify the Docker installation using docker --version.
7. Execute docker info to check the Docker engine configuration and system information.
8. Download the official Ubuntu image from Docker Hub using docker pull ubuntu.
9. Wait until the image download process is completed.
10. Execute docker images to verify that the Ubuntu image is available locally.
11. Create and start a container using docker run -it --name mycontainer ubuntu bash.
12. Observe that an interactive shell is opened inside the Ubuntu container.
13. Execute a simple command such as echo "Hello Docker" inside the container.
14. Execute other basic Linux commands such as pwd, ls and whoami to verify the container environment.
15. Exit the container using the exit command.
16. Return to the host terminal and execute docker ps -a to display all containers, including stopped containers.
17. Identify the container named mycontainer in the displayed list.
18. Restart the stopped container using docker start -ai mycontainer.
19. Verify that the interactive shell is available again.
20. Exit the container and confirm that the container can be started and stopped successfully.

Program / Commands

Docker commands: docker --version docker info docker pull ubuntu docker images docker run -it --name mycontainer ubuntu bash Run inside the container: echo "Hello Docker" exit On the host: docker ps -a docker start -ai mycontainer

Output of Program

Docker Image Download: $ docker pull ubuntu Status: Downloaded newer image for ubuntu:latest Container Execution: $ docker run -it --name mycontainer ubuntu bash root@container:/echo "Hello Docker" Hello Docker root@container:/exit Container Verification: $ docker ps -a CONTAINER ID IMAGE STATUS xxxxxxxx ubuntu Exited Explanation: Docker images act as templates used to create containers. A container is an isolated execution environment created from an image. In this experiment, an Ubuntu image is downloaded and used to create an interactive container.

Result

Thus, Docker was successfully installed and configured. The Ubuntu image was downloaded from Docker Hub, a container was created and started, and basic commands were executed successfully inside the container.

EXPERIMENT 12: Run a Container from Docker Hub

Aim

To download an application image from Docker Hub, create and execute a Docker container using the downloaded image and access the running application through a web browser.

Procedure

1. Start the computer and open a web browser.
2. Navigate to https://hub.docker.com/ to access Docker Hub, which is a public registry for Docker images.
3. Install Docker Desktop or Docker Engine and start the Docker service.
4. Open a terminal or command prompt and verify the installation using docker --version.
5. Search for the Nginx image using the command docker search nginx.
6. Identify the official Nginx image from the search results.
7. Download the image using docker pull nginx.
8. Verify that the image has been downloaded successfully using docker images.
9. Create a container using the downloaded Nginx image.
10. Execute the container in detached mode using docker run -d --name mynginx -p 8080:80 nginx.
11. The port mapping connects port 8080 on the host computer to port 80 inside the container.
12. Verify that the container is running using docker ps.
13. Check the container status and confirm that it is in the running state.
14. Open a web browser on the host computer.
15. Enter the address http://localhost:8080 in the browser's address bar.
16. Verify that the default Nginx welcome page is displayed.
17. This confirms that the Nginx web server is running successfully inside the Docker container.
18. Stop the container using docker stop mynginx.
19. If required, remove the stopped container using docker rm mynginx.
20. Verify the final container status using docker ps -a.

Program / Commands

Docker commands: docker --version docker search nginx docker pull nginx docker images docker run -d --name mynginx -p 8080:80 nginx docker ps Open http://localhost:8080 in a browser docker stop mynginx docker rm mynginx

Output of Program

Docker Container Creation: $ docker run -d --name mynginx -p 8080:80 nginx Container Verification: $ docker ps CONTAINER ID IMAGE STATUS xxxxxxxx nginx Up Browser Output: Welcome to nginx! If you see this page, the nginx web server is successfully installed and working. Explanation: Nginx is a web server that can serve web pages and handle HTTP requests. Docker runs Nginx inside an isolated container. Port mapping allows the host computer to access the web server running inside the container.

Result

Thus, the official Nginx image was successfully downloaded from Docker Hub. A Docker container was created and executed in detached mode, and the Nginx welcome page was accessed successfully through the host computer's web browser.

EXPERIMENT 13: Transfer Files from One Virtual Machine to Another Virtual Machine

Aim

To establish network communication between two Linux virtual machines and transfer files securely from one virtual machine to another using SSH and the SCP command.

Procedure

1. Install Oracle VirtualBox or VMware Workstation on the host computer.
2. Create two virtual machines named VM1 and VM2 using the virtualization software.
3. Install a Linux operating system on both virtual machines.
4. Start both virtual machines and log in using their respective user accounts.
5. Configure both virtual machines to use the same reachable network, such as a host-only network or bridged network.
6. Verify the network configuration and ensure that both virtual machines can communicate with each other.
7. Open the terminal in VM1 and install the OpenSSH server package using the package manager.
8. Repeat the OpenSSH installation process in VM2.
9. Execute hostname -I in VM1 to identify its IP address.
10. Execute hostname -I in VM2 to identify its IP address.
11. From VM1, use the ping command with VM2's IP address to test network connectivity.
12. If the connection is successful, create a sample text file named sample.txt in VM1.
13. Enter sample data into the file using a text editor or the echo command.
14. Execute the SCP command from VM1 to copy the file to the home directory of the user in VM2.
15. Enter the destination user's password if password authentication is enabled.
16. Wait until the file transfer process is completed successfully.
17. Log in to VM2 and navigate to the destination directory.
18. Execute the ls command to verify that sample.txt has been transferred.
19. Execute cat sample.txt to display the contents of the transferred file.
20. Compare the file contents with the original file in VM1 to confirm successful transfer.

Program / Commands

Install SSH and identify addresses: sudo apt update sudo apt install openssh-server -y hostname -I ping Create and transfer a file from VM1: echo "File Transfer Successful" > sample.txt scp sample.txt username@ :/home/username/ Verify on VM2: ls cat sample.txt

Output of Program

VM1 – File Transfer Command: $ scp sample.txt user@192.168.56.102:/home/user/ sample.txt 100% transferred VM2 – File Verification: $ ls sample.txt Displaying File Contents: $ cat sample.txt File Transfer Successful Explanation: SCP stands for Secure Copy Protocol. It uses SSH to transfer files securely between networked computers. In this experiment, VM1 acts as the source machine and VM2 acts as the destination machine.

Result

Thus, two Linux virtual machines were successfully configured on the same network. Network connectivity was established using SSH, and a sample file was securely transferred from VM1 to VM2 using the SCP command.

EXPERIMENT 14: Install a C Compiler in a Virtual Machine and Execute a File Read/Write Program

Aim

To install and configure the GCC compiler in a Linux-based virtual machine and develop, compile and execute a C program to perform file write and read operations using standard file handling functions.

Procedure

1. Start the host computer and open Oracle VirtualBox or VMware Workstation.
2. Start a virtual machine with an installed Linux operating system.
3. Log in to the Linux virtual machine using the configured user account.
4. Open the Terminal application to execute the required Linux commands.
5. Update the package repository using sudo apt update.
6. Install the GCC compiler using sudo apt install gcc -y.
7. Verify the compiler installation using gcc --version.
8. Create a C source file named fileoperation.c using a text editor.
9. Enter the C program to perform file writing and reading operations.
10. Include the required header file and declare a file pointer using the FILE data type.
11. Use the fopen() function with write mode to create or open the file sample.txt.
12. Use fgets() to read the input text from the user and fprintf() to write the data into the file.
13. Close the file using fclose() to ensure that the written data is properly saved.
14. Reopen the same file using fopen() in read mode.
15. Use fgets() inside a loop to read the file contents.
16. Display the retrieved contents on the terminal using printf().
17. Close the file after completing the read operation.
18. Save the C source file and compile it using gcc fileoperation.c -o fileoperation.
19. Execute the generated program using ./fileoperation.
20. Enter a sample text when prompted and observe the write confirmation message.
21. Verify that the contents written into the file are displayed correctly.
22. Use the ls command to confirm that sample.txt has been created in the current directory.

Program / Commands

Install GCC: sudo apt update sudo apt install gcc -y gcc --version Program – fileoperation.c: #include int main() { FILE *fp; char text[100]; // Write operation fp = fopen("sample.txt", "w"); if (fp == NULL) { printf("File cannot be opened"); return 1; } printf("Enter text to write into file: "); fgets(text, sizeof(text), stdin); fprintf(fp, "%s", text); fclose(fp); printf("\nData written successfully.\n"); // Read operation fp = fopen("sample.txt", "r"); if (fp == NULL) { CCS335 – Cloud Computing Lab Page 24 printf("File cannot be opened"); return 1; } printf("\nContents of the file:\n"); while (fgets(text, sizeof(text), fp) != NULL) { printf("%s", text); } fclose(fp); return 0; } Compile and execute: gcc fileoperation.c -o fileoperation ./fileoperation

Output of Program

Sample Input: Enter text to write into file: Cloud Computing Lab Expected Output: Data written successfully. Contents of the file: Cloud Computing Lab Created file: sample.txt Explanation: The program performs two major file operations: - Write Operation: The fopen() function opens the file in write mode, and fprintf() stores the entered data. - Read Operation: The file is reopened in read mode, and fgets() retrieves the stored contents for display. The fclose() function is used to close the file after each operation.

Result

Thus, GCC was successfully installed and configured in the Linux virtual machine. The C program was compiled and executed successfully to perform file write and read operations, and the stored contents were displayed correctly.

EXPERIMENT 15: Write a Google App Engine Program to Display the Nth Largest Number from a List and Deploy It to Google Cloud

Aim

To develop a Python-based web application using the Flask framework to identify and display the Nth largest number from a user-provided list and deploy the application using Google App Engine.

Procedure

1. Open a web browser and navigate to the Google Cloud Console.
2. Sign in using a valid Google account and create a new Google Cloud project.
3. Select the created project and enable the required Google Cloud services.
4. Enable billing if it is required for the selected deployment configuration.
5. Install Python and the Google Cloud CLI on the local computer.
6. Open a terminal or command prompt and execute gcloud init to initialize the Google Cloud CLI.
7. Authenticate using the Google account and select the appropriate Google Cloud project.
8. Create a new application directory named NthLargest.
9. Inside the directory, create a Python source file named main.py.
10. Import Flask and the request module to handle browser requests.
11. Create a Flask application instance and define a route for the home page.
12. Create an HTML form that accepts a comma-separated list of numbers and the value N.
13. Retrieve the submitted values using the Flask request object.
14. Split the comma-separated input into individual values and convert them into integers.
15. Validate the input to ensure that all entered values are valid integers.
16. Validate N to ensure that it is greater than or equal to 1 and does not exceed the number of elements in the list.
17. Sort the list in descending order using the sorting function.
18. Select the element at index N - 1 to identify the Nth largest element.
19. Display the calculated result on the browser using an appropriate HTML response.
20. Create a file named requirements.txt and include Flask as the application dependency.
21. Create an app.yaml file and specify the supported Python runtime.
22. Save all the application files in the same project directory.
23. Open the terminal inside the application directory and execute gcloud app deploy.
24. Wait until the deployment process completes successfully.
25. Execute gcloud app browse to open the deployed application.
26. Enter a sample list of numbers and a value for N.
27. Verify that the application displays the correct Nth largest number.

Program / Commands

File – main.py: from flask import Flask, request app = Flask(__name__) @app.route("/") def largest(): numbers = request.args.get("numbers") n = request.args.get("n") if numbers is None or n is None: return ''' Nth Largest Number Enter numbers (comma separated): Enter N: ''' try: arr = [int(x.strip()) for x in numbers.split(",")] n = int(n) if n len(arr): CCS335 – Cloud Computing Lab Page 26 return "Invalid N value" arr.sort(reverse=True) result = arr[n - 1] return f" {n}th Largest Number = {result} " except ValueError: return "Enter valid numbers" if __name__ == "__main__": app.run(host="0.0.0.0", port=8080) File – requirements.txt: Flask File – app.yaml: runtime: python313 Deployment commands: gcloud init gcloud app deploy gcloud app browse

Output of Program

Sample Input: Input numbers: 10, 50, 30, 20, 40 N = 2 Expected Browser Output: 2nd Largest Number = 40 Explanation: The program sorts the entered numbers in descending order: 50, 40, 30, 20, 10 The second element in the sorted list is 40. Therefore, the second largest number is 40. The application validates the input and displays an error message if the entered numbers or N value are invalid. Note: The program treats repeated values as separate elements. For example, in the list [50, 50, 40], the second largest element is also 50 under this convention.

Result

Thus, a Python web application was successfully developed using Flask to identify the Nth largest number from a given list. The application was deployed on Google App Engine and tested through a web browser, producing the expected result. Quick Revision – Important Commands Experiment Important commands GCC installation sudo apt update, sudo apt install gcc -y, gcc --version C compilation gcc file.c -o program, ./program Hadoop installation java -version, hadoop version Hadoop services hdfs namenode -format, start-dfs.sh, start-yarn.sh, jps Google App Engine gcloud init, gcloud app deploy, gcloud app browse Docker installation docker --version, docker info Docker image docker pull ubuntu, docker images Docker container docker run -it ubuntu bash, docker ps -a Nginx container docker run -d --name mynginx -p 8080:80 nginx VM file transfer scp sample.txt user@ :/home/user/ File verification ls, cat sample.txt Important Points to Remember for the Practical Examination 1. Virtual Machines: A hypervisor creates and manages virtual machines by allocating physical hardware resources to each virtual environment. 2. GCC: GCC is a compiler that converts C source code into executable machine code. 3. Fibonacci Series: Each term is obtained by adding the previous two terms. 4. Google App Engine: It is a Platform as a Service (PaaS) that allows developers to deploy applications without directly managing the underlying server infrastructure. 5. Hadoop: HDFS provides distributed storage, while YARN manages resources and job execution. 6. CloudSim: It is a simulation toolkit used to model cloud infrastructure, virtual machines, cloudlets and scheduling algorithms. 7. Docker Image: An image is a read-only template used to create containers. 8. Docker Container: A container is an isolated environment in which an application and its dependencies can execute. 9. Guest Additions: VirtualBox Guest Additions provides improved integration between host and guest operating systems. 10. SSH and SCP: SSH provides secure remote access, while SCP transfers files securely between networked systems. 11. Flask: Flask is a lightweight Python web framework used to develop web applications. 12. App Engine Deployment: The main.py, requirements.txt and app.yaml files should be maintained in the application directory. 13. Hadoop Configuration: Java and Hadoop environment variables must be configured correctly before starting Hadoop services. 14. Docker Port Mapping: Port mapping allows applications running inside containers to be accessed through ports on the host computer. 15. File Handling in C: fopen(), fprintf(), fgets() and fclose() are commonly used for file operations in C. Exam tip: For every experiment, clearly write the Aim, number the Procedure steps, include the complete program or commands, show a sample input and output, and finish with a Result beginning with “Thus…”. This keeps the answer consistent and easy to evaluate.